
Dr. Pooja's Physio - Finance Admin Test Plan

The finance desk: the books, and the sessions it may read but never change

Role

Finance Admin

Document version

1.0

Audience

A tester checking this role end to end

Environment

Throwaway Supabase project · Razorpay test mode · npm run dev

Read first

Section 1 - who this is, and what has to exist before you start

Contents

1. Who this is, and what must exist first
2. Signing in, and what the screen says you are
3. Today
4. Sessions at [view](#) - the whole of the middle level
5. Money - manage, and the books are the job
6. People - manage, with the money controls that are theirs
7. What Finance must never reach
8. Suspension, and the scope itself
9. Sign-off

1. Who this is, and what must exist first

The desk. Finance keeps the books: revenue, costs, payouts, refunds, discounts and the seven standard finance figures. They can **read** sessions and cannot change one - and that single `view` grant is the most interesting thing in this document, because it is the only place in the product where a desk is deliberately given a section it must not write to.

What this desk opens, exactly.

Section	Finance
Today	manage
Sessions	view
People	manage
Money	manage
Catalog	none
Logs	none
Settings	none

Why `view` exists at all. The question finance actually asks - *what was this ₹1,200 for?* - was answerable only by asking somebody else. Handing them the section outright would let the person reconciling the books cancel the sessions they are reconciling. There is deliberately **no** "write only": a dashboard cannot let somebody change a row they are not allowed to see, so the third box a permissions matrix usually draws is one this product has no honest meaning for.

What makes `view` real. Every admin route guarded by a scope check asks for **manage**. So a section granted at `view` is read-only at **all** of them, without a single one being edited and without any screen remembering to hide a button. This plan proves that at the screen and at the route.

The account.

Label	Email	Password
Finance	<code>qa.admin.finance@example.test</code>	<i>generated when the Master Admin creates it</i>

1.1 Before you start

Must exist	Set up by
This Finance account, created from Settings → User Access	Master Admin
Several paid and completed sessions, online and home visit	Patient + Therapist
One session refunded , and one refund awaiting cash by hand	Master Admin
A therapist with a revenue share, holding cash , and owed a payout	Therapist
A partner hospital on a revenue share with a completed referral	Hospital
At least one programme purchase and one cash-on-visit home visit	Patient
A promo code with at least one claim	Master Admin

The cash matters. A therapist holding cash and owed a payout is what makes the netting arithmetic at **FI-19** testable, and that is the single most expensive bug this desk could miss.

1.2 How to record a result

Mark every step **Pass**, **Fail**, **Blocked** or **N/A**. Raise a failure with: the step id, what you did, what you expected, what happened, a screenshot, any console error in full, how many times out of how many it reproduced, and **which account you were signed in as**.

Severity. P0 - money, clinical data or access control is wrong: stop and report. **P1** - a main journey is broken with no workaround. **P2** - wrong with a workaround. **P3** - cosmetic.

1.3 Calling a route without a terminal

The **view** grant is only real at the routes, so a good half of this document is console work. Sign in as Finance, press **F12**, open **Console**, paste and press Enter:

```
const r = await fetch("/api/some/route", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ some: "value" }),
});
({ status: r.status, body: await r.text() });
```

The request is same-origin, so the browser attaches this admin's cookie itself. Use a **second browser profile** for a different account, not a second tab. Never paste the URL in the address bar: that sends a GET, these are POSTs, and you will get 405.

2. Signing in, and what the screen says you are

FI-01 - The dashboard names itself · P2

Do this. Sign in as the Finance admin.

Expect

- The sidebar brand reads **Finance**, and the **eyebrow over the section heading** reads **Finance** too. **Both. A dashboard that just says "Admin Panel" is a P2.**
- A **"Your access" card** naming which sections this desk covers. A shorter sidebar with no explanation reads as a fault.

FI-02 - The sidebar · P1

Expect present: Today, Sessions, People, Money.

Expect absent: Catalog, Logs, Settings.

Sessions is present - **view** is enough to open a section. That is the point: what **view** costs them is every control on it, not the section itself.

3. Today

FI-03 - Finance leads with what is owed · P1

Expect

- The headline figure is **what is owed to therapists** - this desk's own first question. Operations leads with unassigned sessions and Clinical with the recommendation queue.
- That is **emphasis**, not permission: nothing reachable is hidden further down.

FI-04 - An unassigned session is not their work · P1

Do this. Read the "needs a person" figure and the queue list beneath it.

Expect

- **Unassigned sessions are not counted.** A queue is a piece of work, and Finance reads Sessions without being able to assign one - so counting an unassigned session here would put a figure on their screen that nothing they could do would ever bring down. **P1 if it is counted.**
- Sessions they **can** work appear; sessions they can only **read** do not produce work.
- The figure and the list **agree**.

This is the sharpest test of the difference between **view** and **manage** anywhere in the product: the section is open, and it still holds no work.

FI-05 - Quick actions land inside this scope · P1

Do this. Tap each quick action and come back.

Expect

- Every one lands on a screen Finance can open.
- **None lands back on Today.** A link into a section they cannot open falls back silently to the first one they can - **a quick action that reloads Today is a P1.**
- A quick action that would **change** a session is **dropped**, not rendered and then refused.

FI-06 - Activity is their own desk's · P2

Do this. Open **Today** → **Activity**.

Expect entries whose action belongs to a section this desk can **work** and whose **actor** sits at this desk. It runs **sparse**, and the screen **says the list is their desk's** rather than showing an empty panel that reads as "nothing happened". **P2 if it just looks empty.**

FI-07 - Risk, the money rules only · P2

Do this. Open **Today** → **Risk**.

Expect

- The rules whose domain is **money**: a cash variance, a session completed with no payment.
- **Not** the clinical ones - a contact leak is a sessions question.
- **The flagged messages panel and the contact reveal log are absent.** Those quote what a colleague wrote and name every patient contact a therapist opened. **Either rendering here is a P0.**

- **No action buttons on the tab.** A flag is never an accusation.
- Review a money signal with a real note: **succeeds**. With `ok`: refused.

4. Sessions at `view` - the whole of the middle level

This section is the reason this document exists. Work it carefully.

`FI-08` - The section opens, and reads properly · P1

Do this. Open **Sessions** → **All Sessions**.

Expect

- The list renders in full, with filters, paging and the date range.
- **The amount and the refund are readable** - Finance is precisely the desk that may see money. This is the opposite of Operations and Clinical, whose exports and screens omit both.
- Open a session's detail drawer: the payment, the refund state, the discount facts and the travel fee are all there.
- Open **Sessions** → **Delivery**: no-show, cancellation and repeat-booking rates render.

`FI-09` - Every control that changes a session is gone · P1

Do this. Look at a session's detail drawer, and at a row in the list.

Expect absent

Control	
Assign / Tap to assign	☐
Reschedule	☐
Cancel	☐
Complete	☐
Reopen	☐
Edit booking	☐

A control an admin's scope cannot call must not render, or they get a 403 with nothing on screen to explain it. **Each one that renders is a P1.**

And the two on the patient's profile. Open **People** → **Patients**, open a patient, and look at their session list: the **Complete** and **Reopen** buttons must not be there either.

FI-10 - The routes refuse, which is what makes `view` real · P0

```
const routes = [
  "/api/admin/assign-appointment",
  "/api/admin/update-appointment",
  "/api/admin/cancel-appointment",
  "/api/admin/create-booking",
  "/api/admin/reopen-session",
  "/api/admin/assign-referral",
  "/api/admin/save-therapist-availability",
  "/api/admin/set-availability-exception",
  "/api/admin/set-therapist-on-leave",
  "/api/admin/review-care-plan",
  "/api/admin/edit-and-approve-care-plan",
  "/api/admin/author-care-plan",
  "/api/admin/withdraw-care-plan",
];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 90) };
})));
```

Expect 403 on every one. These are all POSTs that change something, so every one of them asks for `manage` and Finance has `view`.

Any 200 is a P0 that stops the run. The whole meaning of `view` is that it holds at all of them without one being edited.

FI-11 - Completing a session is the one they must not reach · P0

This route is shared between a therapist and an admin, so it cannot simply guard on the section - it has to tell "an admin who may not" from "not an admin at all".

```
const r = await fetch("/api/appointments/complete-session", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ appointmentId: "PASTE_A_PAID_SESSION_ID" }),
});
({ status: r.status, body: await r.text() });
```

Expect a refusal.

A Finance admin completing a session is a P0, and it is the most consequential one in this document. `completed` **and** `paid` is the exact and only condition that makes a therapist's revenue share payable - so completing a session **creates the payout obligation**. Finance holds Sessions at `view` precisely so the person reconciling the books cannot create the obligation they are then asked to settle, and this route is exempt from both of the gates a therapist faces.

FI-12 - The roster reads and does not write · P1

Do this. Open **Sessions → Roster**.

Expect. It opens and reads. **No save control, no exception control, no leave switch.** The routes above already covered it; confirm the screen agrees.

FI-13 - Recommendations read and do not decide · P1

Do this. Open **Sessions** → **Recommendations**.

Expect. The queue renders. **No Approve, no Turn down, no Approve with changes, and no panel offering to write one on a therapist's behalf.** All four are **manage**.

5. Money - manage, and the books are the job

FI-14 - Summary, and the two identities · P0

Do this. Open **Money** → **Summary**.

Expect both identities hold, every time:

- **net = gross – refunds**
- **clinic share = splittable net – therapists' share – partners' share**

Either failing is a P0.

And the three rules behind the split:

Rule	Check
A therapist's share is earned by delivering	A paid session not yet completed adds nothing . Counting every paid one deducts a share nobody will be paid and understates the clinic's take on every forfeited late cancellation
A home visit's travel is the therapist's, never revenue	Confirm travel is not in the revenue lines
A refund reverses the partner's commission, not the therapist's	A refunded session was cancelled, so it never earned a therapist share

- A session whose split is **unknowable** - no therapist share set, or a partner with none configured - is **excluded from the split and surfaced as a named count. A guessed percentage to make the numbers tie is a P0.**
- **Each figure appears once on the screen.** Net revenue printed twice is a **P2**.

FI-15 - Open a total · P2

Do this. Tap **See the sessions** on each of the four split figures.

Expect. Each lists **exactly the rows behind it**, and those rows **sum to the figure that opened them. A drill-down that disagrees with its own total is a P1** - it makes a correct figure look wrong.

The drill-down exports like every other table: CSV and PDF, same rows.

FI-16 - Scope chips: flows move, balances do not · P1

Do this. Narrow the date range and watch each figure.

Expect

- Flows move: revenue, refunds, what was settled in the period.

- "Owed to therapists" **does not**. It is **all-time and net of cash held**, matching the Payouts screen and what the Pay button actually transfers. **A real debt reading as nothing because you narrowed to a quiet week is a P1.**
- Every figure carries a **scope chip** - `range` , `now` or `setting` - so watching one fall while the one beside it holds still does not read as a half-broken screen.

FI-17 - The alerts strip · P1

Expect it at the top of **all** the Money screens, counting: payout requests waiting, cash a therapist is holding, refunds to hand back by hand, and payments attached to nothing.

Check	Expect
Refunds awaiting cash	Counts both cash home visits and sessions refunded by hand
Cash home visits	Counted once , not twice. The home-visit rows are the same table filtered, not a second table - summing the two puts a figure on the strip that the ledger beneath it disagrees with
A failed refund	Not on this strip. It is fixed on Sessions , which Finance can only read - so counting it here would put a figure on their screen that nothing they could do would bring down
Each item	Links to the rows it counted
A zero row	Dropped , not shown as a zero

That failed-refund rule is the same one as `FI-04` , in a second place. **A failed refund counted on the Finance strip is a P2.**

FI-18 - Costs · P1

Do this. Add three costs, one of each kind:

Description	Amount	Kind
<code>QA Clinic rent - month 1</code>	<code>40000</code>	fixed
<code>QA Physiotherapy consumables</code>	<code>6000</code>	variable
<code>QA Loan interest</code>	<code>3500</code>	interest

Expect

- Each carries a **kind**, deciding whether it sits above or below the gross-profit line, whether it is inside break-even's fixed costs, and whether EBITDA adds it back. **Nothing is inferred from the wording.**
- The **gateway fee** is derived from what was collected **online**, charged on **gross** - a processor keeps its fee through a refund - and **skipped for cash on visit**, which never touched a gateway.
- It is labelled **Gateway fee %** here, a rate, distinct from the resulting amount Summary calls Payment fees. **Two figures under one name is a P2.**
- Costs are dated by **when they were incurred**, not when somebody typed them.
- **Discounts given** is **stated, never deducted**. It is already inside gross revenue as a smaller number; subtracting it from profit would count it twice and understate profit by exactly the amount given away. **A discount deducted from operating profit is a P0.**
- With no costs recorded, **Operating profit is a ceiling and the screen says so.**
- Nothing here is labelled "net profit", and nothing is post-tax.

- **The promo-code campaigns are on this screen**, beside **Discounts given** - the figure they cost. **P2 if they are somewhere else**, because following a note that says they are here and finding no promo codes is a dead end.

FI-19 - Payouts, and the netting that must happen once · P0

Do this. With a therapist holding cash and owed a share:

1. Open **Money → Payouts**.
2. Settle it.
3. Read the cash ledger.
4. Settle a **second** payout for the same therapist later.

Expect

- The transfer is the earned figure **minus the cash they are holding**, and **exactly those visits are marked remitted in the same run**.
- Step 4 **does not deduct the same cash again. A second deduction is a P0** - the same rupees taken twice, and it asks somebody to chase money already recovered.
- Where the cash **exceeds** what is owed, the transfer **floors at zero**, the difference stays as still-owed-to-the-clinic, and those collections stay **open on the cash ledger** for a person to chase. It does not go negative.

FI-20 - Correcting a cash amount · P1

The person holding the cash does not get to say how much of it the clinic knows about; this is the lane that belongs to somebody who is not holding it.

Try	Expect
Correct a cash amount with a reason of ok	Refused
Correct it with a real reason	Succeeds , with an audit row
Correct one whose cash has already been remitted	Refused . That transfer has gone out, so the fix is an adjustment against the next payout rather than a silent edit of a settled one

FI-21 - Business Health: a refusal beats a zero · P0

Do this. Open **Money → Business Health** with **nothing** entered on Your Numbers.

Expect

- Seven figures: return on investment, return on ad spend, working capital, gross and net margin, EBITDA, break-even, revenue run rate.
- **A figure that cannot be worked out is a sentence naming the missing input, with a link to the screen that takes it - never a zero.** A zero is read as a measurement and acted on. **A zero here is a P1.**
- Revenue and the split come from the **same functions Summary reads**, so the two screens **cannot disagree about what the clinic earned**. Compare them figure by figure.
- Every (i) carries the **formula and where each input came from**, not merely what the word means.
- **Working capital** counts money taken for sessions not yet delivered as a **current liability**, valued at **what was actually paid** and never at the live catalogue price.

- Move the three cost-classification switches. **Gross margin moves; operating income and net profit do not.** A switch that moved the bottom line would be a way to report a different profit: **P0**.
- Apply a **dimension filter** - one therapist. An **amber line** says the comparison is one slice's revenue against the whole clinic's costs, because rent is not attributable to a therapist. **Silently filtered profit is a P1.**

FI-22 - Your Numbers - P1

Do this. Enter an investment with a life in months, an ad campaign traced by a promo code, some untraceable ad spend, and a dated snapshot.

Expect

- The **life in months** produces the depreciation and amortization inside EBITDA.
- A campaign is traced **by promo code or not at all**. Untraceable spend is **stated separately and held out of the division** - leaving it in the denominator reports a campaign as a failure purely because nobody tagged it.
- A **hand-entered** revenue figure is allowed and is **labelled as the owner's own** wherever it shows.
- Spend is **pro-rated across the campaign's own days**, and an open-ended one runs to **today** - never to the end of the range in view, or one row would read as a different daily budget every time you moved the dates.
- Snapshots sharing an **as of** date are **one snapshot**, and the most recent at or before the range end is read. Entering this month's bank balance does **not** erase last month's.

FI-23 - Discounts, from the books' side - P0

Do this. Open a discounted booking and read what was recorded.

Expect all four facts on every one: **list price, amount off, which rule,** and **why.**

A discount implemented by simply charging less leaves the books unable to tell "we sold this cheap" from "we discounted it" - and that difference is the one number that decides whether an offer continues. **A missing fact is a P1.**

Then check the rules themselves from this desk:

Check	Expect
Two discounts both apply	One applies - the largest - and the loser is released so it does not count against its own cap for nothing
A tie	Goes to the more deliberate decision: goodwill, then the typed code, then the campaign that runs itself
A home visit	The discount is on the service line only ; travel is added back undiscounted . Discounted travel is a P0
A 100% discount	The total is zero , with no gateway order and no `payments` row - and never a ₹1 charge
A goodwill amount at or above the session price	Refused
Apply goodwill, then collect by cash	The recorded cash is the discounted figure with the list price beside it. Recording the full price overstates the cash ledger and gross revenue by exactly the amount given away: P0

FI-24 - The money glossary, and one word per figure · P2

On every Money screen:

1. **Two lines under the heading** - what the screen is in a clinic owner's words, and one concrete example of something you would come here to do.
2. The **glossary** at the foot, and an **(i)** on each figure. The glossary is the fallback for reading the whole set; the (i) is the answer, because a definition at the bottom of the screen is as far from the number as the page allows.
3. **No word does two jobs.** "Package cash collected" is what came into the bank up front; recognised revenue is that same money earned one session at a time. **Two figures meaning the same thing is a P2** - delete one rather than explaining the difference.

6. People - manage, with the money controls that are theirs

FI-25 - The directories · P1

Do	Expect
Open People → Patients , open one	Their sessions, purchases and payments - with the money readable
Approve a pending signup	Succeeds
Suspend an account	Succeeds
Reset a password	Succeeds , shown once, never in the log
Look at the Complete and Reopen buttons on a session row	Absent - those are Sessions writes

FI-26 - Partners, and a control that is theirs · P1

Do this. Open **People** → **Partners** and open Hospital A.

Expect

- **The revenue-share editor and the two share figures DO render.** Unlike Operations and Clinical, changing a revenue share is a **money** route and Finance holds Money at manage. **Their absence here is a P1.**
- Change it: **succeeds**, with an audit row.
- The partner's commission on the Money screens moves to match.

This is the mirror image of the Operations case, and worth checking in both documents: the section is the same, the control is the same, and the two desks must answer differently.

FI-27 - Delete and impersonation are still full scope only - P0

```
const routes = [
  "/api/admin/delete-account",
  "/api/admin/start-impersonation",
];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 90) };
})));
```

Expect a refusal on both, and no control for either on screen. Both are checked against **full scope directly** rather than through People, precisely so a section scope cannot hand them out. **A 200 on impersonation is a P0 that stops the run.**

7. What Finance must never reach

FI-28 - Catalog, at the screen and at the routes - P0

Try	Expect
Find Catalog in the sidebar	Absent
Open the Catalog URL	Refused, with a dismissible line saying their scope is why and naming a screen that exists

```
const routes = [
  "/api/admin/create-treatment-category",
  "/api/admin/update-treatment-category",
  "/api/admin/delete-treatment-category",
  "/api/admin/reorder-treatment-categories",
  "/api/admin/create-package",
  "/api/admin/update-package",
  "/api/admin/create-home-visit-package",
  "/api/admin/create-home-visit-areas",
  "/api/admin/upload-catalog-image",
];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 90) };
})));
```

Expect 403 on every one. Finance must not be able to re-price the catalogue - which would change what future patients pay - from the desk that reports on what they paid.

But note the split. The **refund** and **credit** controls that live on Catalog → Purchases are **money** routes, so those are Finance's:

```
const mine = [
  "/api/admin/refund-package",
  "/api/admin/grant-session-credits",
  "/api/admin/reverse-session-credit",
  "/api/admin/revive-entitlement",
];
console.table(await Promise.all(mine.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 90) };
})));
```

Expect a validation error, not 403 - the empty body is wrong, but the caller is allowed. **403 here is a P1**: the section is chosen by the capability, not by where the button happens to sit, and a refund is money.

FI-29 - Logs, at the screen and at both routes · P0

Try	Expect
Find Logs in the sidebar	Absent
Open the Logs URL	Refused, with the scope line

```
const routes = ["/api/admin/activity-log", "/api/admin/clear-activity-log"];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 90) };
})));
```

Expect 403 on both. Logs is Master Admin's alone - the record of who settled which payout, among other things. A desk that settles payouts reading and clearing the record of its own settlements is exactly the separation this refusal keeps. **A 200 on `clear-activity-log` is a P0.**

Finance reads their own desk's work on **Today → Activity** instead.

FI-30 - Settings, at the screen and at the routes · P0

Try	Expect
Find Settings in the sidebar	Absent
Open Settings → Offers & Discounts by URL	Refused
Open Settings → User Access by URL	Refused

```
const routes = [
  "/api/admin/update-setting",
  "/api/admin/set-admin-scope",
  "/api/admin/set-admin-active",
  "/api/admin/create-account",
  "/api/admin/debug-reset",
];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 90) };
})));
```

Expect 403 on every one.

`update-setting` matters most here. The first-session offer and the invite rewards are **settings**, not Money screens - so Finance can report what discounting cost and **cannot change what the clinic gives away. A 200 here is a P0.**

`create-account` is the escalation case: the Back office group is absent from their picker, but that is presentation - the full-only check inside the route is what stops a limited scope minting itself a Master Admin.

Note the one that is theirs. The promo-code switch sits on **Money → Costs**, beside the campaigns it governs, rather than in Settings - because an admin who has just written a code and cannot see why it does nothing is the failure that placement avoids. Confirm `save-promo-code` and `delete-promo-code` **succeed** for Finance.

FI-31 - The other roles' doors · P0

```
const routes = [
  "/api/therapist/reveal-contact",
  "/api/therapist/record-cash-collection",
  "/api/patient/condition-profile/export",
  "/api/hospital/withdraw-referral",
];
console.table(await Promise.all(routes.map(async (route) => {
  const r = await fetch(route, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: "{}",
  });
  return { route, status: r.status, body: (await r.text()).slice(0, 90) };
})));
```

Expect a refusal on every one. `record-cash-collection` in particular: the figure a therapist asserts nets straight off what they owe, and a finance desk asserting it on their behalf would be the books writing their own inputs.

8. Suspension, and the scope itself

FI-32 - Suspending this admin ends the session · P0

Do this

1. **As the Master Admin**, suspend the Finance admin.
2. In the Finance admin's **still-open browser**, reload the dashboard.
3. In the same browser, settle a payout from the console.
4. Un-suspend.

Expect

- Step 2: bounced.
- Step 3: **refused**. A suspended admin's own routes refuse them too - the session was ended, not merely redirected away from. **A suspended finance admin still settling payouts is a P0**: money would leave the clinic on the authority of an account somebody closed.
- Step 4 restores them to **Finance**, not to full and not to a default.

FI-33 - Nobody changes their own scope · P0

```
const r = await fetch("/api/admin/set-admin-scope", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ userId: "YOUR_OWN_ID", scope: "full" }),
});
({ status: r.status, body: await r.text() });
```

Expect a refusal. This landing is a P0 and the most serious finding this document can produce.

9. Sign-off

	Should be
Steps run	33
Pass	
Fail	
Blocked	
N/A	
P0 raised	
P1 raised	

Confirm each of these before signing.

The dashboard says Finance , twice, and carries the access card	☐
--	--------------------------

Four sidebar entries; Catalog, Logs and Settings absent	☐
Today leads with what is owed, and does not count unassigned sessions	☐
Sessions opened and read in full , money included	☐
Every session-changing control absent from the screen	☐
403 on all thirteen session-changing routes	☐
`complete-session` refused	☐
Both money identities held, on Summary and Business Health alike	☐
A balance did not move when the date range did	☐
A payout settled once, with the cash netted once	☐
Business Health refused rather than printing a zero	☐
A cost switch moved gross margin and not the bottom line	☐
All four discount facts recorded, travel never discounted, zero never ₹1	☐
The partner revenue-share editor did render, and changing it worked	☐
403 on the catalogue routes, and not on the refund and credit routes	☐
403 on both log routes, <code>clear-activity-log</code> included	☐
403 on <code>update-setting</code> and <code>create-account</code> ; promo codes still theirs	☐
Suspension ended the session; the scope came back as Finance	☐
They could not widen their own scope	☐

Signed **Date**

Build / commit tested

What a clean run of this document proves. That the Finance scope grants what the grid says it grants and refuses what it says it refuses - in particular that Sessions at `view` is read-only at every route and not merely on the screens that remembered to hide a button - on one machine, once. It does not prove the money figures themselves are right against a real ledger, and it does not cover the other two desks, each of which has a document of its own.